인공지능과 기계학습

1주차

2026.03.04

장 우 영

목 차

1. 오리엔테이션
2. 컴퓨터는 데이터에서 배운다
3. 간단한 분류 알고리즘 훈련

오리엔테이션

- 수업 구성
- 교재
- 강의자료
- 평가방법
- 질의 응답

- 수업 구성

교시	시간	주요 내용
10	18:00 ~ 18:50	복습 / 질의응답
11	19:00 ~ 19:50	강의 / 실습
12	20:00 ~ 20:50	강의 / 실습
13	21:00 ~ 21:50	예습 / 질의응답

주차	수업내용	교재범위 및 과제물	비고
1	머신러닝 개요와 퍼셉트론 기초 이해	주교재 1장, 2장	
2	주요 분류 알고리즘 구조와 차이 이해	주교재 3장	
3	데이터 전처리와 차원 축소 방법 이해	주교재 4장, 5장	
4	모델 평가 기법과 앙상블을 통한 성능 향상	주교재 6장, 7장	
5	회귀 분석과 연속값 예측 모델 이해	주교재 9장	
6	군집 분석과 텍스트 데이터 분석 기초	주교재 8장, 10장	
7	중간시험		
8	다층 신경망과 역전파 원리 이해	주교재 11장	
9	PyTorch 기반 신경망 구현 방법 학습	주교재 12장, 13장	
10	CNN과 RNN을 활용한 딥러닝 응용 이해	주교재 14장, 15장	
11	Transformer와 어텐션 메커니즘 이해	주교재 16장	
12	GAN을 통한 생성 모델 이해	주교재 17장	
13	그래프 신경망(GNN) 구조 이해	주교재 18장	
14	강화학습과 MDP, Q-learning 이해	주교재 19장	
15	기말시험		

오리엔테이션

- 교재

요리	머신러닝
장보기	데이터 모으기
재료 손질	데이터 정리하기
레시피 정하기	어떤 방법으로 배울지 결정
요리해보기	모델 만들기
맛보기	결과 확인
간 맞추기	틀린 부분 수정
다시 요리	다시 학습
손님에게 내놓기	실제 사용

주방 공간	python
(기본 도구) 칼, 도마	numpy, pandas
(전문기기) 자동 온도 조절 오븐	Pytorch, tensorflow

오리엔테이션

- 강의자료

- https://github.com/rickiepark/ml-with-pytorch
- https://github.com/WooyeongJang/ml-with-pytorch

오리엔테이션

- 평가방법

- 중간고사와 기말고사 단답형 시험
- 과제제출 1Page
- 기타 (수업참여)
- 출석 75% 미만 F

오리엔테이션

- 질의 응답

1장 컴퓨터는 데이터에서 배운다

1.1 데이터를 지식으로 바꾸는 지능적인 시스템 구축
1.2 머신 러닝의 세가지 종류
1.3 기본 용어와 표기법 소개
1.4 머신 러닝 시스템 구축 로드맵
1.5 머신 러닝을 위한 파이썬
1.6 요약

1장 요리사는 재료에서 배운다

1.1 재료만 주고 요리사 육성
1.2 요리사 육성 방법
1.3 요리의 기본 표기법
1.4 요리사를 훈련시키는 전체 과정
1.5 주방 준비하기
1.6 요약

학습 내용

- 머신 러닝의 주요 개념과 종류
- 관련 용어 소개

1.1 데이터를 지식으로 바꾸는 지능적인 시스템 구축

- 데이터를 분석해 의미 있는 규칙을 찾아내고,
  그 규칙으로 문제를 해결하는 시스템
  - 예전에는 사람이 규칙을 직접 만들었지만,
  - 머신러닝은 데이터를 많이 주면 컴퓨터가 스스로 규칙을 찾는데
  - 이게 바로 인공지능의 한 방식

학습 방식
: 지도학습, 비지도학습, 강화학습

모델 구조
: 선형모델, 결정트리, SVM, 신경망

1.2 머신 러닝의 세가지 종류

- 지도학습
  - 레이블된 데이터
  - 직접 피드백
  - 출력 및 미래 예측
- 비지도학습
  - 레이블 및 타깃 없음
  - 피드백 없음
  - 데이터에서 숨겨진 구조 찾기
- 강화학습
  - 결정 과정
  - 보상시스템
  - 연속된 행동에서 학습

1.2.1 지도 학습으로 미래 예측하기

- 지도(supervised)는 희망하는 출력 신호(레이블)가 있는 샘플을 의미

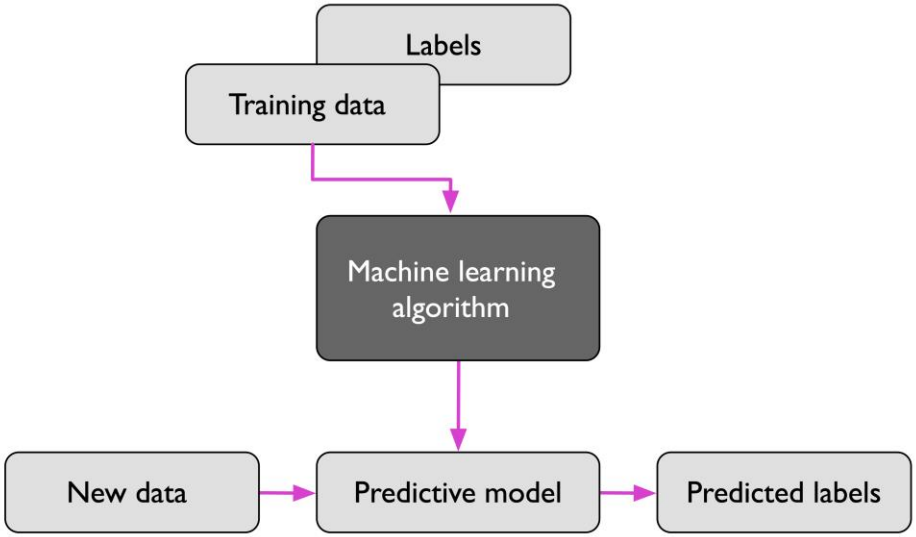
- 메일 필터링: 스팸 또는 스팸이 아닌 이메일

: 2개의 범주 중 어디에 속하는지

: 분류

- 지도 학습의 또 다른 종류는 연속적인 값을 예측하는

: 회귀



회귀

- Regression은 라틴어 "다시 돌아가다"
  - re- : 다시
  - gress : 가다

- 19세기 통계학자 "극단 값이 평균으로 되돌아가는 현상"
  : 수학적도구: regression analysis

- Regression을 일본에서 回帰 (かいき, 카이키)
  回 : 돌 회
  帰 : 돌아갈 귀

1.2.1 지도 학습으로 미래 예측하기

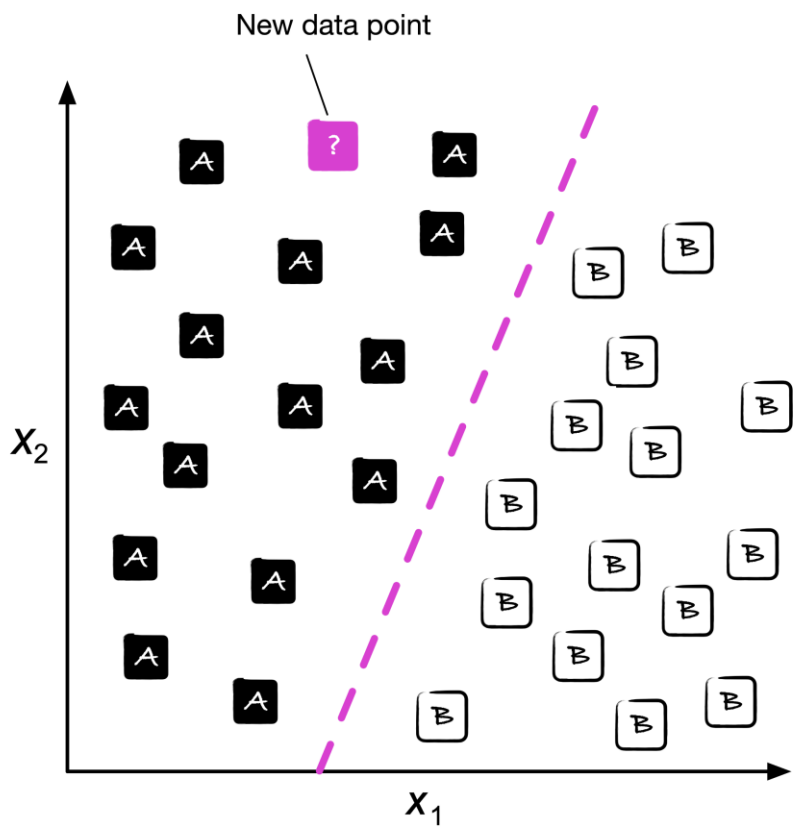
- 분류: 클래스 레이블(범주형 출력 값) 예측
  - 이진 분류/ 다중 분류
  - 양성 클래스/ 음성 클래스
  - 결정 경계

구분	분류	회귀
출력값	범주(카테고리)	연속된 숫자
예	고양이/개	집값 3억 2천
예	A/B/C 등급	온도 23.7도

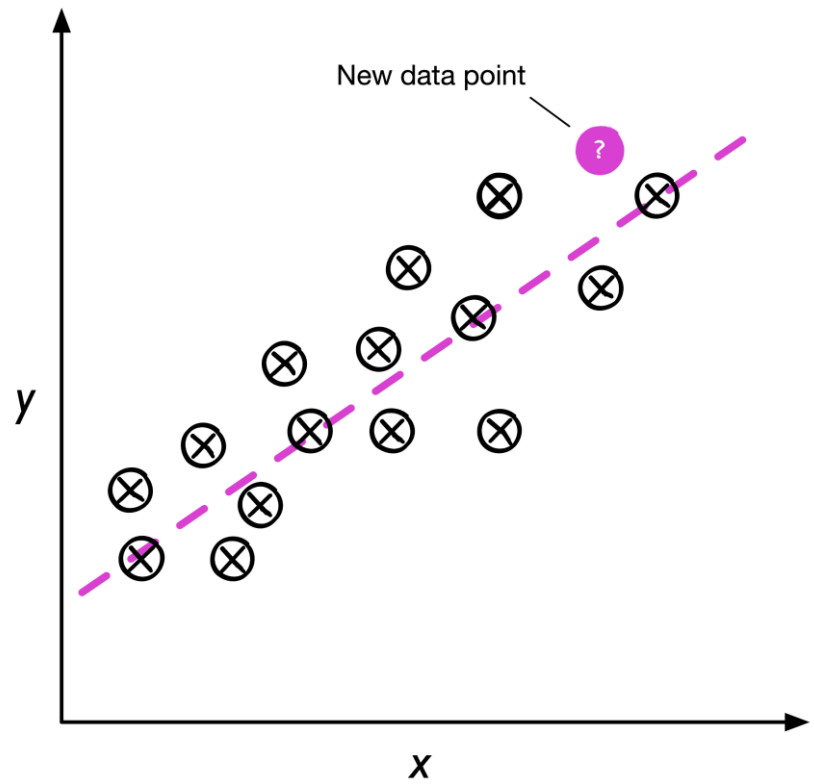
- 회귀: 연속적인 출력 값 예측
  - 예측 변수(특성) vs 반응 변수(타깃)
  - 선형 회귀

Q. 집값을 예측하는데 분류를 이용할 수 있을까?

1.2.1 지도 학습으로 미래 예측하기



분류

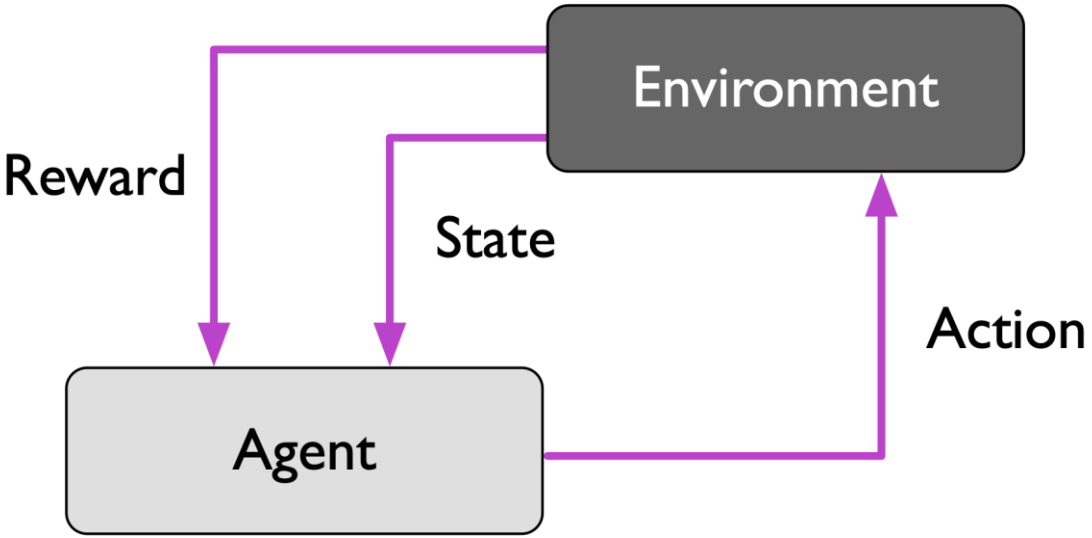


회귀

1.2.2 강화 학습으로 반응형 문제 해결

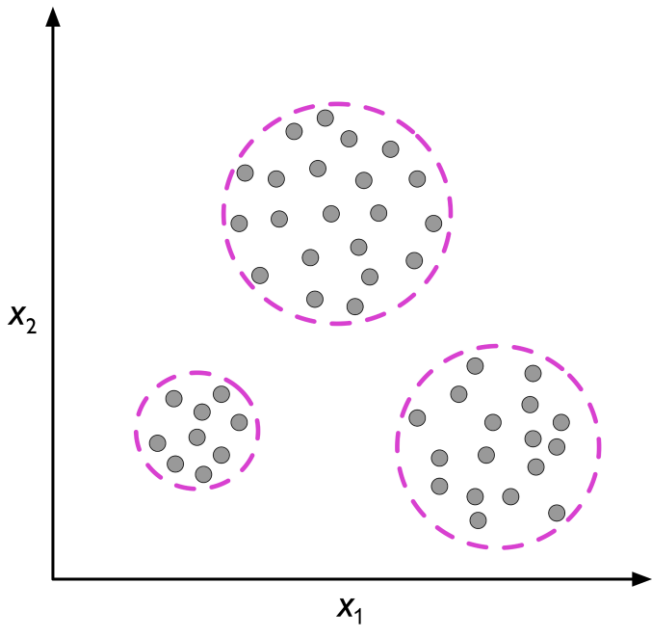
환경과 상호작용하여 학습 하는 방법
: 체스게임

- Reward
- Action
- State
- Agent
- Environment

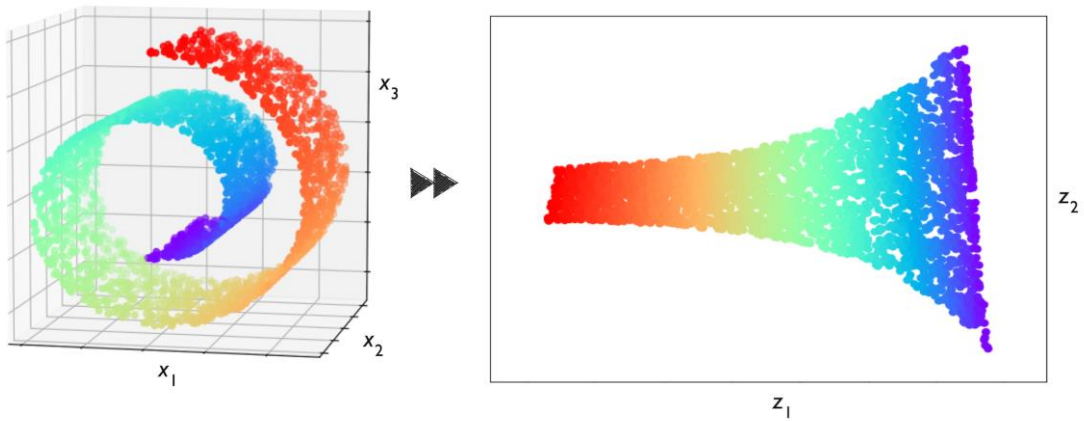


1.2.3 비지도 학습으로 숨겨진 구조 발견

- 군집: 서브그룹 찾기
- 차원 축소: 데이터 압축



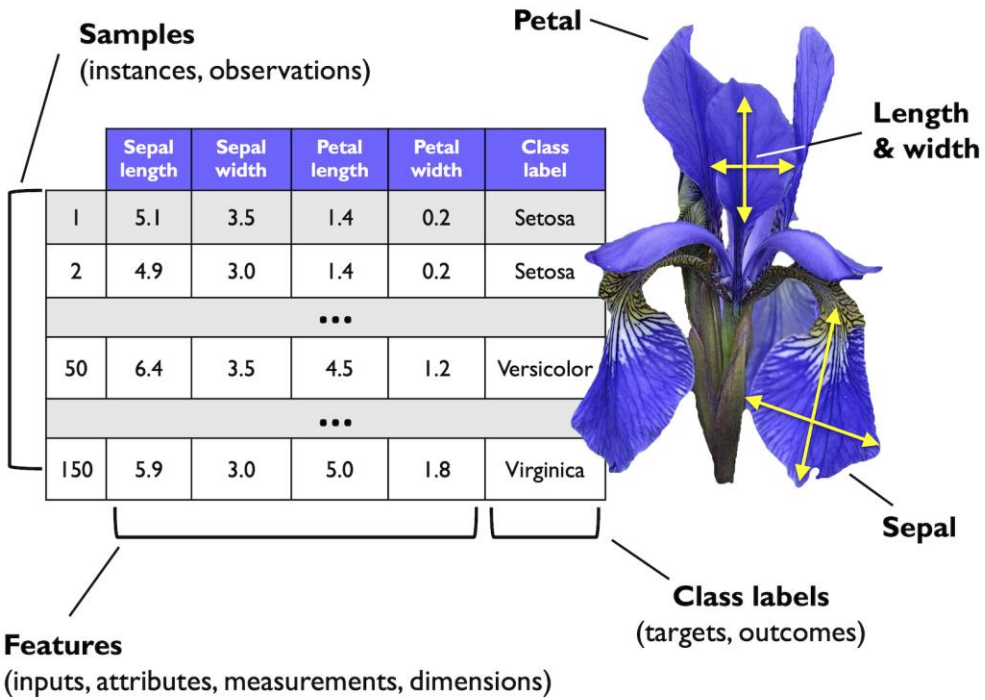
군집
(비지도 분류)



차원 축소

1.3 기본 용어와 표기법 소개

- 책에서 사용할 기본 용어
- 수학 표기법
- 고전적 예제: 붓꽃 데이터



	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
...					
50	6.4	3.5	4.5	1.2	Versicolor
...					
150	5.9	3.0	5.0	1.8	Virginica

1.3.1 이 책에서 사용하는 표기법과 규칙

- 샘플은 특성 행렬 X에 있는 행으로 나타내고, 특성은 열을 따라 저장
- 150개 샘플과 네 개의 특성을 가진 붓꽃 데이터셋은 150 X 4 크기의 행렬

$$\begin{pmatrix} x_1^{(1)} & x_2^{(1)} & x_j^{(1)} & x_4^{(1)} \\ x_1^{(2)} & x_2^{(2)} & x_j^{(2)} & x_4^{(2)} \\ x_1^{(i)} & x_2^{(i)} & x_j^{(i)} & x_4^{(i)} \\ \vdots & \vdots & \vdots & \vdots \\ x_1^{(150)} & x_2^{(150)} & x_j^{(150)} & x_4^{(150)} \end{pmatrix}$$

1.3.1 이 책에서 사용하는 표기법과 규칙

Q. $x_1^{(150)}$

A. 150번 째 샘플에 첫번째 특성

Q. $\mathrm{x}^{(150)} = [x_1^{(150)} x_2^{(150)} x_3^{(150)} x_4^{(150)}]$

A. 150번 샘플이 갖는 특성

Q. $\mathrm{x}_1 = \begin{bmatrix} x_1^{(1)} \\ x_1^{(2)} \\ x_1^{(\mathrm{i})} \\ \vdots \\ x_1^{(150)} \end{bmatrix}$

A. 첫 번째 특성을 갖는 150개의 샘플

1.3.1 이 책에서 사용하는 표기법과 규칙

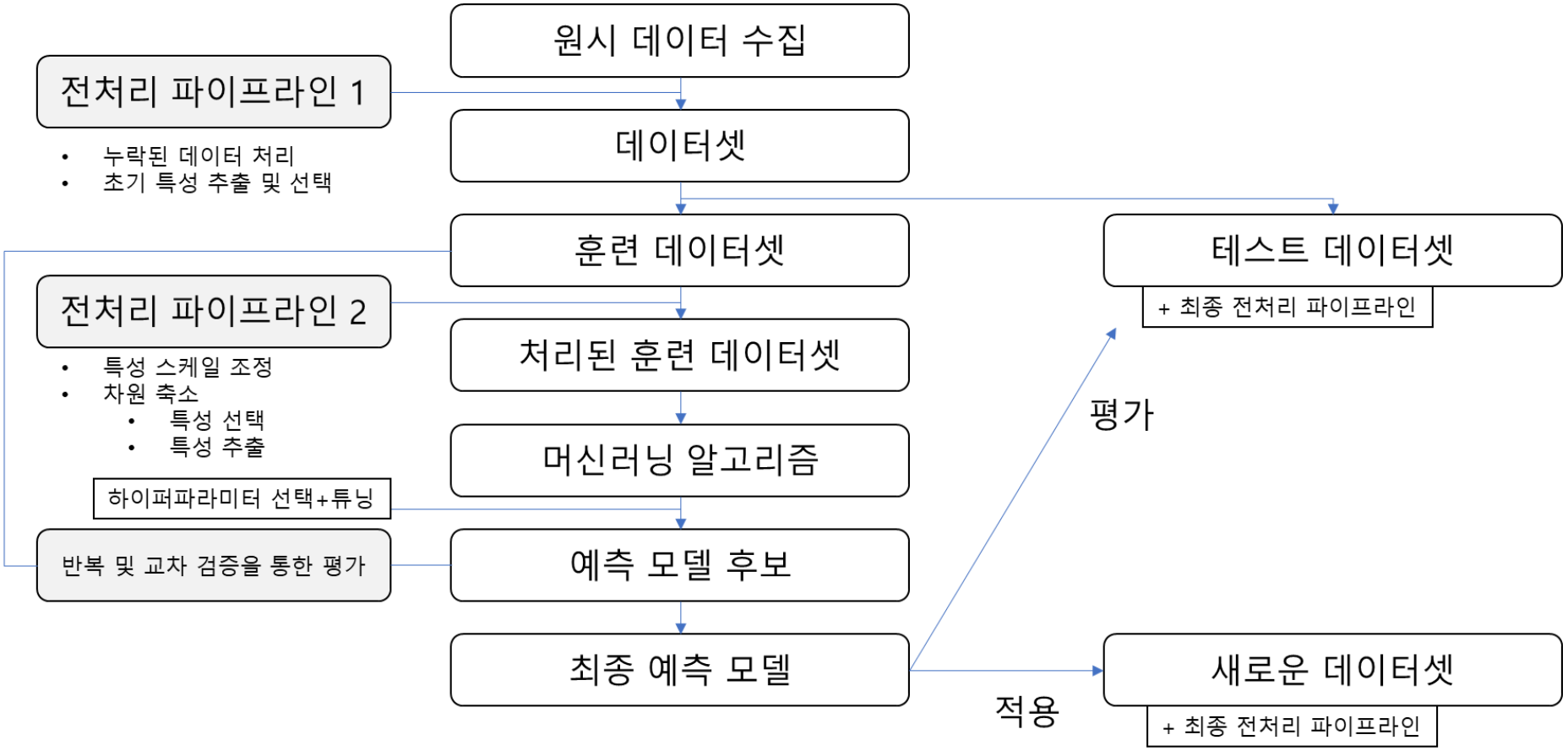
- 타깃 변수(클래스 레이블)

$$y = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ \vdots \\ y^{(150)} \end{pmatrix} \qquad y \in \{\text{Setosa, Versicolor, Virginica}\}$$

1.3.2 머신 러닝 용어

- 훈련 샘플: 데이터셋을 나타내는 테이블의 행, 관측, 레코드, …
- 훈련: 모델 피팅, 파라미터 추정
- 특성(x): 변수, 입력
- 타깃(y): 출력, 반응 변수, 정답
- 손실 함수: 하나의 데이터 포인트에 대해 특정한 손실
- 비용 함수: 전체 데이터셋에 대해 계산한 손실

1.4 머신 러닝 시스템 구축 로드맵



1.4.1 전처리: 데이터 형태 갖추기

- 원본 데이터의 형태가 학습 알고리즘에 적합한 경우는 적음
- 데이터 전처리는 애플리케이션 적용을 위한 중요한 단계

1.4.2 예측 모델 훈련과 선택

- **데이비드 월퍼트**(David H. Wolpert)**가 주도한**
  "No Free Lunch (NFL, **공짜 점심 없음**) **이론**
- 모든 알고리즘의 평균 성능은 같다
- 보편적 최적 알고리즘은 없다
- 일반화 능력은 문제의 구조/가정에 좌우됨

- "가진 도구가 망치밖에 없다면 모든 문제가 못으로 보일 것이다"

1.4.2 예측 모델 훈련과 선택

- 모든 음식에 항상 가장 맛있게 만드는 “만능 레시피”는 없다.
  - 굽기 (Grilling)
  - 삶기 (Boiling)
  - 오븐 베이킹

- 모든 요리가 같은 확률로 나온다고 가정한다.
  - 디저트, 한식, 생식, 국물요리, 냉요리, 생고기, 수프, 튀김 (특정 조리법이 항상 우월할 수 없다)
  - 하지만 현실에서는 “일식 오마카세” , “한식 오마카세“ …

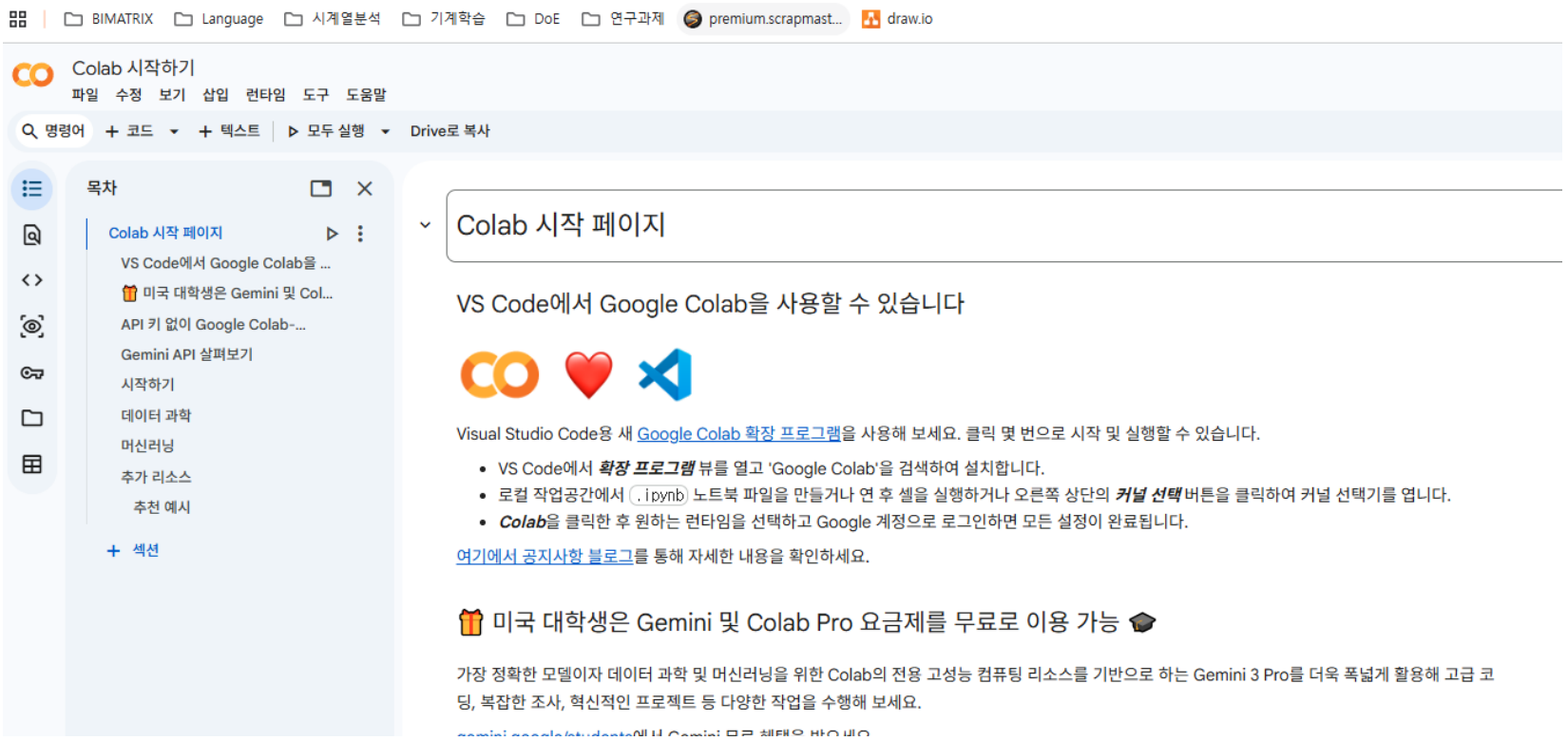
1.4.3 모델을 평가하고 본 적 없는 샘플로 예측

- 훈련데이터에서 최적의 모델을 선택한 후 테스트 데이터셋을 사용하여 본 적이 없는 데이터에서 얼마나 성능을 내는지 예측하여 일반화 오차를 예상

- 이전에 언급한 특성 스케일 조정과 차원 축소 같은 단계에서 사용한 파라미터는 훈련데이터셋만 사용하여 얻은 것
  - 나중에 동일한 파라미터를 테스트 데이터셋은 물론 새로운 모든 샘플을 변환하는데 사용

1.5 머신 러닝을 위한 파이썬

- https://colab.research.google.com



1.5 머신 러닝을 위한 파이썬

- [하드웨어 가속기] GPU / Quantum
- [운영체제] 윈도우/ 맥OS/ 리눅스
- [프로그래밍 언어] Python/ R
- [라이브러리/프레임워크] PyTorch, pandas, etc.
- [자연어] LLM + Orchestrator
- [사용자]

1.6 요약

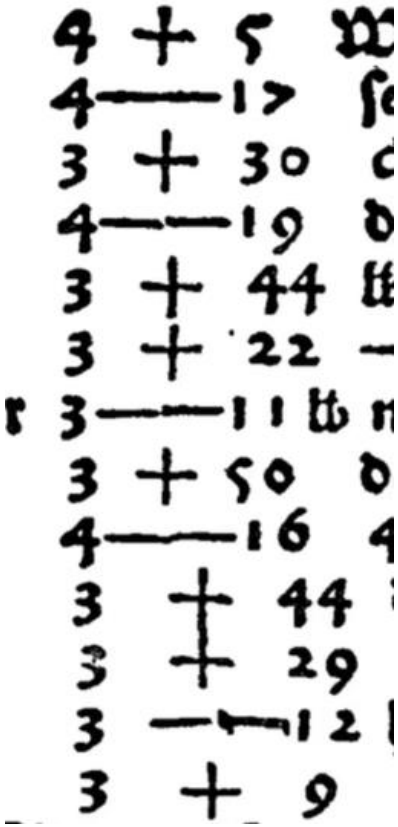
- 큰 그림 이해: AI > ML > DL
- 지도학습: 분류(클래스 예측), 회귀(연속값 예측)
- 비지도학습: 구조 탐색, 차원 축소(데이터 압축)
- 머신러닝 로드맵: 문제정의 → 데이터 → 학습 → 평가

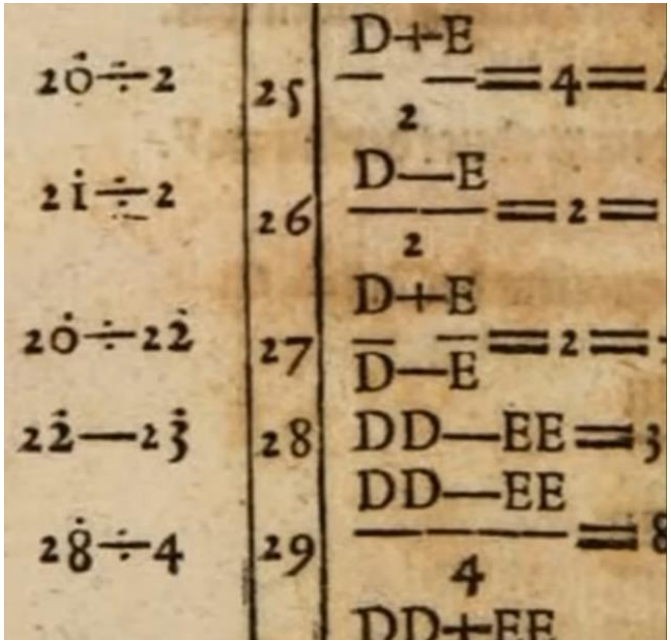
2장 간단한 분류 알고리즘 훈련

2.1 인공 뉴런: 초기 머신 러닝의 간단한 역사
2.2 파이썬으로 퍼셉트론 학습 알고리즘 구현
2.3 적응형 선형 뉴런과 학습의 수렴
2.4 요약

수학기호 누가 만들었을까?

일의 자리 십의 자리 백의 자리 천의 자리





수학기호 = 형식 언어

기호	설명
논리	
$p \vee q$	p 또는 q
$p \wedge q$	p이고 q, p 그리고 q
$\neg p$, $\sim p$	p가 아니다
$p \Rightarrow q$, $p \rightarrow q$	p이면 q
$p \Leftrightarrow q$, $p \leftrightarrow q$, $p \rightleftarrows q$	p와 q는 동치
$\forall x\ px$	모든 x에 대해 $p(x)$가 성립한다
$\exists x\ p(x)$	$p(x)$가 성립되도록 하는 x가 존재한다
집합	
$a \in A$	a는 집합 A의 원소이다
$a \notin A$, $a \in A$	a는 집합 A의 원소가 아니다
$A \subseteq B$	A는 B의 부분집합이다
$A \subset B$	A는 B의 진부분집합이다 ($A \subseteq B$, $A \subset B$ 대신 각각 $A \subset B$, $A \subseteq B$로 쓰는 경우도 있다)
ϕ	공집합
$A \cup B$	A와 B의 합집합
$\cup A_i$	모든 A_i의 합집합
$A \cap B$	A와 B의 교집합
$\cap A_i$	모든 A_i의 공통부분
A^c, $C(A)$	A의 여집합
$A-B$, $A \setminus B$	A에 대한 B의 차집합
$A \times B$	A와 B의 직곱집합
ΠA_i	모든 A_i의 직곱집합
$\lvert A \rvert$, $\mathrm{Card}(A)$, $\overline{\overline{A}}$	A의 농도
$\aleph_0$	가산집합의 농도
$\aleph$, χ	$\boldsymbol{R}$의 농도($\aleph$는 알레프로 읽는다)
$\{x \mid p(x)\}$	$p(x)$가 성립되도록 하는 x 전체의 집합
$f: A \rightarrow B$	집합 A에서 B로의 함수(사상)
$f: a \rightarrow b$	함수(사상) f에 의해 원소 a에 원소 b가 대응한다
f^{-1}	f의 역함수(역사상)
$f \circ g$	f와 g의 합성함수(합성사상)
$f(E)$, $f(a)$	f에 의한 집합 E, 원소 a의 상
$f^{-1}(E)$, $f^{-1}(a)$	f에 관한 집합 E, 원소 a의 원상
$\bar{A}$	집합 A의 개포
A^o	집합 A의 개핵
A^d	집합 A의 도함수
수	
$\boldsymbol{N}$	자연수 전체의 집합 $\{1, 2, \cdots\}$ ($\{0, 1, 2, \cdots\}$를 나타내는 경우도 있다)
$\boldsymbol{Z}$	정수 전체의 집합
$\boldsymbol{Q}$	유리수 전체의 집합
$\boldsymbol{R}$	실수 전체의 집합
$\boldsymbol{C}$	복소수 전체의 집합
$\boldsymbol{R}^n$	$\boldsymbol{R} \times \cdots \times \boldsymbol{R}$, n차원유클리드공간
$[a, b]$	폐구간 $\{x \mid a \leq x \leq b\}$
(a, b), $]a, b[$	개구간 $\{x \mid a < x < b\}$
$[a, b)$, $[a, b[$	반개구간 $\{x \mid a \leq x < b\}$
$(a, b]$, $]a, b]$	반개구간 $\{x \mid a < x \leq b\}$

기호	설명
$\max A$	실수의 집합 A의 원소 중 최대인 것
$\max a_n$	실수 a_n 중 최대인 것
$\min A$	실수의 집합 A의 원소 중 최소인 것
$\min a_n$	실수 a_n 중 최소인 것
$\sup A$	실수의 집합 A의 상한
$\inf A$	실수의 집합 A의 하한
$n!$	자연수 n의 계승
$a \mid b$	정수 a는 정수 b의 약수이다
$a \nmid b$	정수 a는 정수 b의 약수가 아니다
$\lvert z \rvert$	복소수 z의 절대값
$\bar{z}$	복소수 z의 켤레복소수
$\Re z$, $\mathrm{Re}\ z$	복소수 z의 실수부분
$\Im z$, $\mathrm{Im}\ z$	복소수 z의 허수부분
$\arg z$, $\mathrm{arc}\ z$	복소수 z의 편각
${}_nP_r$	n개의 물건에서 r개를 빼내 순서 있게 늘어놓은 순열의 수
${}_nC_r$, $\binom{n}{r}$	n개의 물건에서 순서를 고려하지 않고 r개를 택한 조합의 수
기하, 대수	
$\angle ABC$, $\angle B$	반직선 BA와 BC가 이루는 각, B를 꼭지점으로 하는 각
$\angle(h, k)$	반직선 h와 k가 이루는 각
$\angle R$	직각
$l \parallel l'$	직선 l, l'은 평행이다
$l \perp l'$	직선 l, l'은 수직이다
$\triangle ABC = \triangle DEF$	삼각형 ABC와 DEF는 합동이다
$\triangle ABC \backsim \triangle DEF$	삼각형 ABC와 DEF는 닮았다
$\lVert \boldsymbol{a} \rVert$, $\lvert \boldsymbol{a} \rvert$	벡터 $\boldsymbol{a}$의 놈(norm, 길이)
$(\boldsymbol{a}, \boldsymbol{b})$, $\langle \boldsymbol{a}, \boldsymbol{b} \rangle$, $\boldsymbol{a} \cdot \boldsymbol{b}$	벡터 $\boldsymbol{a}$, $\boldsymbol{b}$의 내적(스칼라곱)
$\boldsymbol{a} \times \boldsymbol{b}$	벡터 $\boldsymbol{a}$, $\boldsymbol{b}$의 외적(벡터곱)
${}^tA$, A'	행렬 A의 전치행렬
A^*	행렬 A의 수반행렬
A^{-1}	행렬 A(변환 A)의 역행렬(역변환)
$\mathrm{tr}\ A$	행렬 A의 트레이스
$\det A$, $\lvert A \rvert$	행렬 A의 행렬식
$\dim V$	벡터공간 V의 차원
$V_1 \dotplus V_2$, $V_1 \oplus V_2$	벡터공간 V_1, V_2의 직합
$R_1 \oplus R_2$	환 R_1, R_2의 직합
$V_1 \otimes V_2$	벡터공간 V_1, V_2의 텐서곱
$A \otimes B$	행렬의 텐서곱, 크로네커곱
δ_{ij}, δ_i^j	크로네커의 기호
$a \cup b$	속(lattice), 불대수의 원소 a, b의 매듭
$a \cap b$	속, 불대수의 원소 a, b의 만남
미적분, 해석	
$\lim_{n \to \infty} a_n$	수열 $\{a_n\}$의 극한
$a_n \rightarrow a$	수열 $\{a_n\}$은 a에 수렴한다
$\lim_{n \to \infty} P_n$	점열 $\{P_n\}$의 극한
$P_n \rightarrow P$	점열 $\{P_n\}$은 점 P에 수렴한다
$\lim_{x \to a} f(x)$	$x \rightarrow a$일 때의 함수 f의 극한
$f(x) \rightarrow a$	함수 f는 a에 수렴한다

기호	설명
$\lim_{P \to Q} f(P)$, $\lim_{x \to a,\ y \to b} f(x, y)$	$P(x, y)$가 $Q(a, b)$에 가까울 때의 함수 f의 극한
$\overline{\lim_{n \to \infty}} a_n$, $\limsup_{n \to \infty} a_n$	수열 $\{a_n\}$의 상극한
$\underline{\lim_{n \to \infty}} a_n$, $\liminf_{n \to \infty} a_n$	수열 $\{a_n\}$의 하극한
$\frac{df}{dx}$, f', $\frac{dy}{dx}$, y'	함수 $y=f(x)$의 도함수
$\frac{d^nf}{dx^n}$, $f^{(n)}$, $\frac{d^ny}{dx^n}$, $y^{(n)}$	함수 $y=f(x)$의 n차도함수
$\dot{y}$, $\ddot{y}$	독립변수 t(시간)에 관한 1차 도함수 및 2차도함수
$\frac{\partial f}{\partial x}$, f_x 등	다변수의 함수 f의 편도함수
$\frac{\partial^2 f}{\partial x^2}$, f_{xx}, $\frac{\partial^2 f}{\partial x \partial y}$, f_{xy} 등	다변수의 함수 f의 2차편도함수
df	1변수의 함수 f의 미분, 2변수의 함수 f의 전미분
Δf	함수 f의 계차
$\Delta^n f$	함수 f의 n계 계차
$\int_a^b f(x)\,dx$	함수 f의 정적분
$\int f(x)\,dx$	함수 f의 부정적분
$\iint_A f(x, y)\,dxdy$	함수 $f(x, y)$의 집합 A 위의 2중적분
$\int_C$	곡선 C위의 선적분을 나타낸다
$\oint_C$	폐곡선 C 위의 선적분을 나타낸다
$\iint_S$	곡면 S위의 면적분을 나타낸다
$\oiint_S$	폐곡면 S 위의 면적분을 나타낸다
$\sin x$, $\cos x$, $\tan x$ $\cot x$, $\sec x$, $\mathrm{cosec}\ x$	삼각함수
$\sin^{-1}x$, $\arcsin x$ 등	역삼각함수
$\mathrm{Sin}^{-1}x$, $\mathrm{Arcsin}\ x$ 등	역삼각함수의 주치
$\sinh x$, $\cosh x$ 등	쌍곡선함수
e^x, $\exp x$	지수함수
$\log x$, $\ln x$	로그함수
$\Gamma(x)$	감마함수
$B(x, y)$	베타함수
$\zeta(x)$	제타함수
$\vartheta(x)$	시타함수
$\mathfrak{B}(x)$	베함수
$J_\nu(x)$	베셀함수
∇f, $\mathrm{grad}\ f$	함수 $f(x, y, z)$의 기울기(그레이디언트)
$\nabla \boldsymbol{v}$, $\mathrm{div}\ \boldsymbol{v}$	벡터값함수 $\boldsymbol{v}$의 발산
$\nabla \times \boldsymbol{v}$, $\mathrm{rot}\ \boldsymbol{v}$, $\mathrm{curl}\ \boldsymbol{v}$	벡터값함수 $\boldsymbol{v}$의 회전 (∇는 나블라라고 읽는다)
Δf, $\nabla^2 f$	함수 f의 라플라시안
$\Box \boldsymbol{v}$	벡터값함수 $\boldsymbol{v}$의 달랑베르시안
확률	
$P(A)$, $Pr(A)$	사상 A의 확률
$P_A(B)$, $P(B/A)$	조건 A에서의 사상 B의 확률(조건부확률)
$E(X)$	확률변수 X의 평균(기대값)
$F(x) = P\{w \mid X(w) < x\}$	확률변수 X의 분포함수

기호	읽는 법	뜻
∀	포올	모든
∃	익지스트	존재한다

∀ 는 영어 All에서
∃ 는 Exists에서
사실 영어 약자

학습 내용

- 머신 러닝 알고리즘을 직관적으로 이해하기
- Pandas, Numpy, matplotlib 읽고, 처리, 시각화
- 선형 분류 알고리즘 구현하기

라이브러리	의미	역할
NumPy	Numerical Python	수치 계산, 행렬 연산
Pandas	Panel Data	표 형태 데이터 처리
Matplotlib	MATLAB plotting	데이터 시각화

Panel Data + s구조화된 데이터(표 형태)를 다루는 라이브러리
(Panel Data → 계량경제학에서 쓰는 용어 (시계열 + 단면 데이터)
Numerical Python 파이썬 수치 계산 라이브러리
Python용 MATLAB 스타일 plotting 라이브러리

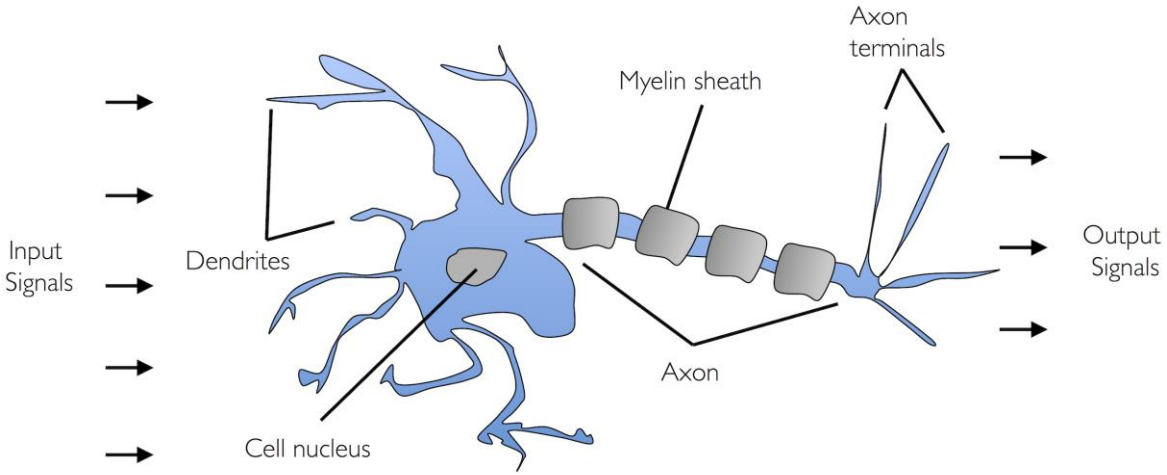
2.1 인공 뉴런: 초기 머신 러닝의 간단한 역사

- 로젠블라트, 퍼셉트론 (Perceptron, 1957, Rosenblatt)
  : 이진 분류를 위한 지도학습 알고리즘
  MCP 뉴런 + 학습 규칙
  학습가능한 최초의 신경망 모델
- 맥컬록-피츠 뉴런 (McCulloch-Pitts Neuron, 1943)
  - 구조 제안: 인공신경망의 출발점, 입력 신호들을 받음, 가중합 계산
  - 임계값(threshold) 넘으면 1, 아니면 0 출력
  - 생물학적 뉴런을 단순화한 수학적 모델

인간처럼 인식 하는(perceive) , 인식(perception) 장치(-tron)

2.1.1 인공 뉴런의 수학적 정의

- 인공 뉴런(artificial neuron) 아이디어
  - 0과 1 두 개의 클래스가 있는 이진 분류 작업
- 입력 값 x와 이에 상응하는 가중치 벡터 w의 선형 조합
  - 결정 함수 σ(z)를 정의
  - 최종 입력(new input)인 z는 $z = w_1x_1 + w_2x_2 + \cdots + w_mx_m$



$$w = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \qquad x = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

2.1.1 인공 뉴런의 수학적 정의

- 특정 샘플 $x^{(i)}$ 의 최종 입력이 사전에 정의 된 임계 값 θ보다 크면 클래스를 1로 예측하고, 그렇지 않으면 클래스 0으로 예측
- 퍼셉트론 알고리즘에서 결정 함수 $\sigma(\cdot)$ 는 단위 계단 함수(unit step function)를 변형한 것

$$\sigma(z) = \begin{cases} 1 & z \geq \theta \text{ 일 때} \\ 0 & \text{그 외} \end{cases}$$

2.1.1 인공 뉴런의 수학적 정의

절편 유닛 b=-θ를 정의하고 z 식에 포함

$$w = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, x = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

$$\mathrm{z} = \mathrm{w}_1\mathrm{x}_1 + \mathrm{w}_2\mathrm{x}_2 + \cdots + \mathrm{w}_\mathrm{m}\mathrm{x}_\mathrm{m} + \mathrm{b}$$

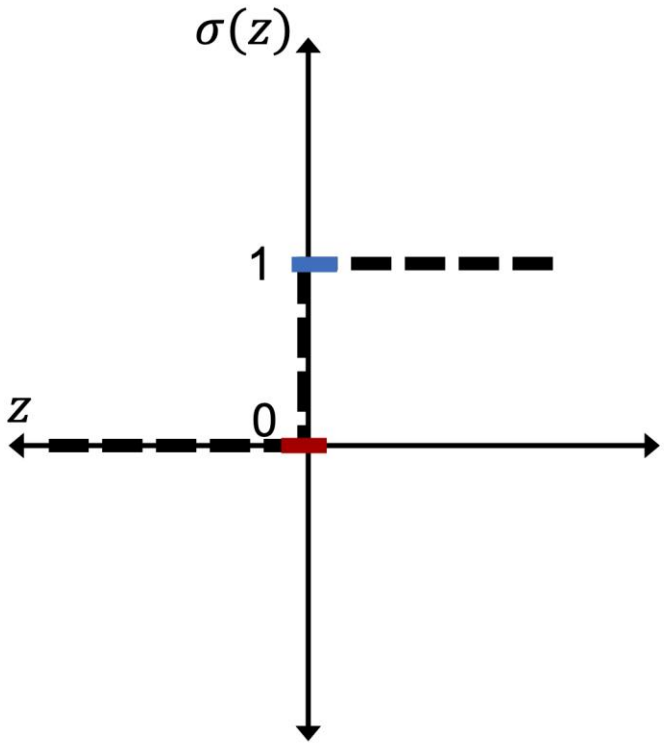
$$\mathrm{z} = \mathrm{w}^\mathrm{T}\mathrm{x} + \mathrm{b}$$

$$\mathrm{y} = \sigma(z)$$

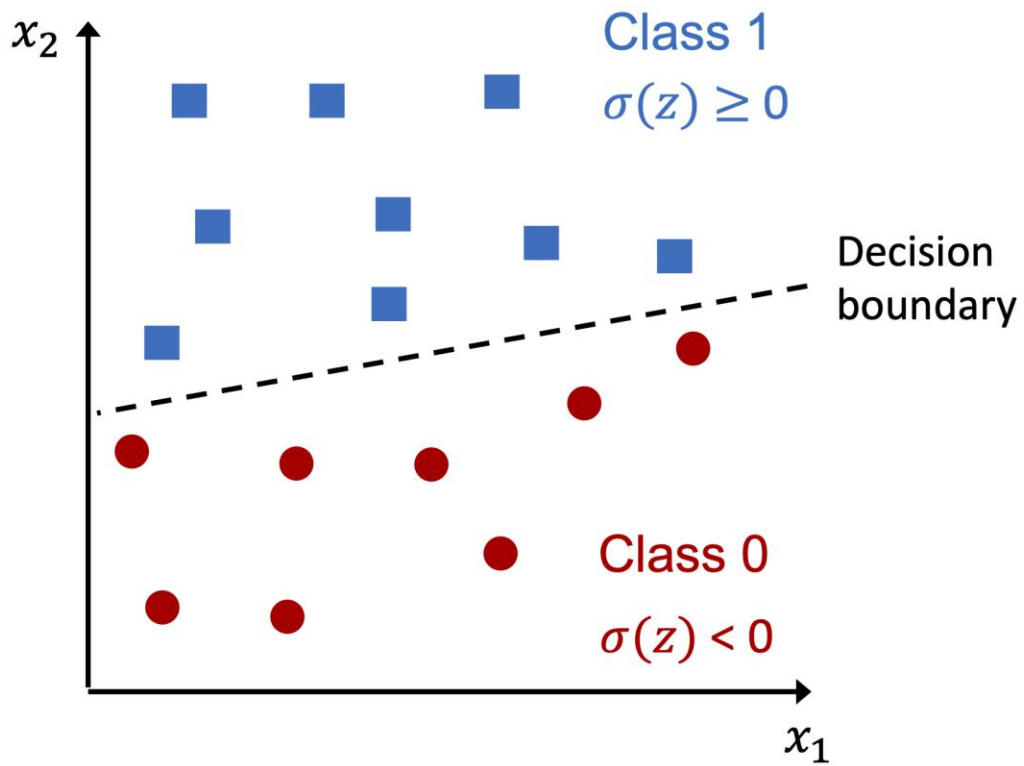
$$\sigma(z) = \begin{cases} 1 & z \geq \theta \text{ 일 때} \\ 0 & \text{그 외} \end{cases}$$

$$\sigma(z) = \begin{cases} 1 & z \geq 0 \text{ 일 때} \\ 0 & \text{그 외} \end{cases}$$

2.1.1 인공 뉴런의 수학적 정의



where $z = \boldsymbol{w}^T\boldsymbol{x} + b$



2.1.2 퍼셉트론 학습 규칙

1. 가중치를 0 또는 랜덤한 작은 값으로 초기화
2. 각 훈련 샘플 $x^{(i)}$에서 다음 작업 수행
    1. 출력 값 $\hat{y}$를 계산
    2. 가중치와 절편을 업데이트

2.1.2 퍼셉트론 학습 규칙

1. 국을 끓이고 먼저 맛을 본다. (예측)

2. 목표 맛과 비교한다. (정답과 비교)

3. 싱거우면 소금을 더 넣고, 짜면 줄인다. (틀렸을 때만 수정)

4. 이 과정을 반복하며 점점 딱 맞는 맛에 가까워진다.

2.1.2 퍼셉트론 학습 규칙

◆ **가중치 업데이트**

$$w_j := w_j + \Delta w_j$$

$$\Delta w_j = \eta \left(y^{(i)} - \hat{y}^{(i)} \right) x_j^{(i)}$$

◆ **바이어스 업데이트**

$$b := b + \Delta b$$

$$\Delta b = \eta \left(y^{(i)} - \hat{y}^{(i)} \right)$$

2차원 데이터셋 예시

$$\Delta w_1 = \eta \left(y^{(i)} - \text{output}^{(i)} \right) x_1^{(i)}$$

$$\Delta w_2 = \eta \left(y^{(i)} - \text{output}^{(i)} \right) x_2^{(i)}$$

$$\Delta b = \eta \left(y^{(i)} - \text{output}^{(i)} \right)$$

$$w_1 := w_1 + \Delta w_1$$

$$w_2 := w_2 + \Delta w_2$$

$$b := b + \Delta b$$

2.1.2 퍼셉트론 학습 규칙

시도	물의양	스프양	목표 맛
1	많음 (3)	적당 (2)	맛있음(1)
2	적음 (1)	많음 (3)	맛없음(0)
3	많음 (3)	많음 (3)	맛있음(1)

① 1번 시도

물=3, 스프=2, 목표맛 y=1

1 가중합

$$z = 0 \cdot 3 + 0 \cdot 2 + 0 = 0$$

2 output

$$output = 1$$

3 오차

$$1 - 1 = 0$$

4 업데이트

변화 없음

$$w_1 = 0,\ w_2 = 0,\ b = 0$$

② 2번 시도

물=1, 스프=3, 목표맛 y=0

1 가중합

$$z = 0 \cdot 1 + 0 \cdot 3 + 0 = 0$$

2 output

$$output = 1$$

3 오차

$$0 - 1 = -1$$

4 업데이트

$$\Delta w_1 = (-1) \cdot 1 = -1$$

$$\Delta w_2 = (-1) \cdot 3 = -3$$

$$\Delta b = -1$$

새로운 값:

$$w_1 = -1, \quad w_2 = -3, \quad b = -1$$

③ 3번 시도

물=3, 스프=3, 목표맛 y=1

1 가중합

$$z = (-1) \cdot 3 + (-3) \cdot 3 + (-1)$$

$$z = -3 - 9 - 1 = -13$$

2 output

$$output = 0$$

3 오차

$$1 - 0 = 1$$

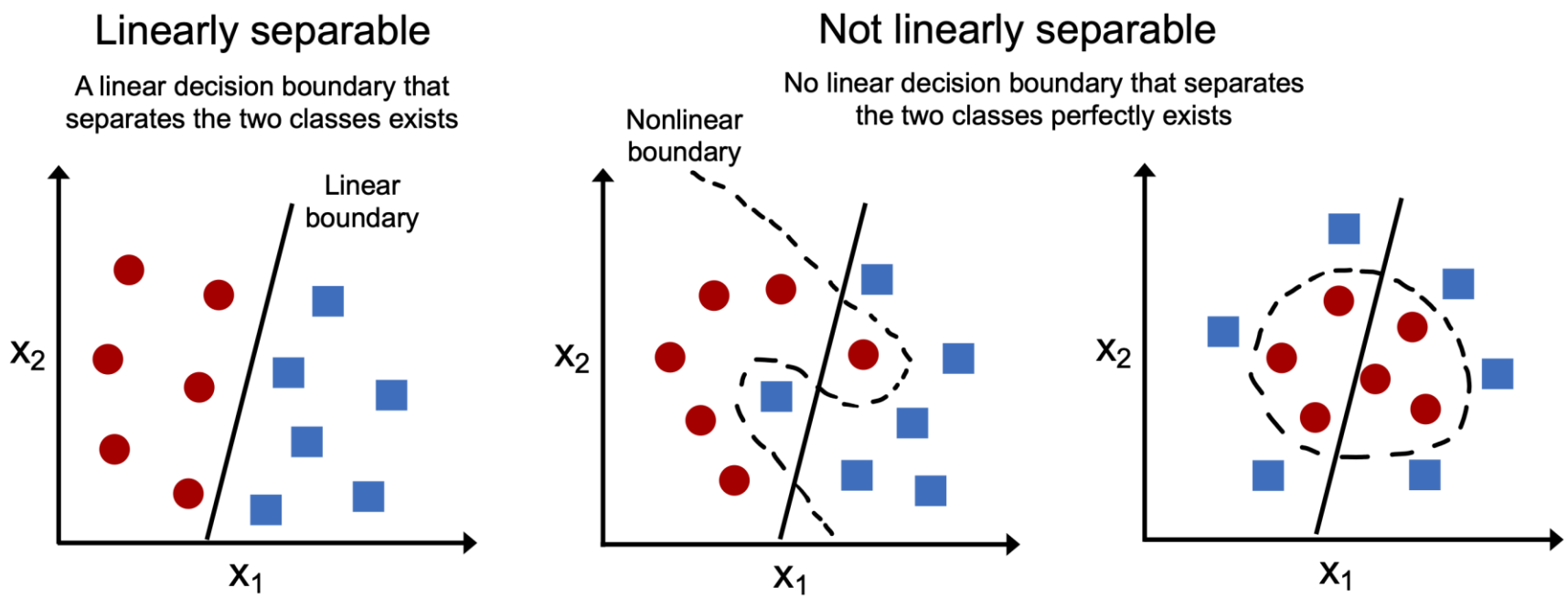
4 업데이트

$$\Delta w_1 = 1 \cdot 3 = 3$$

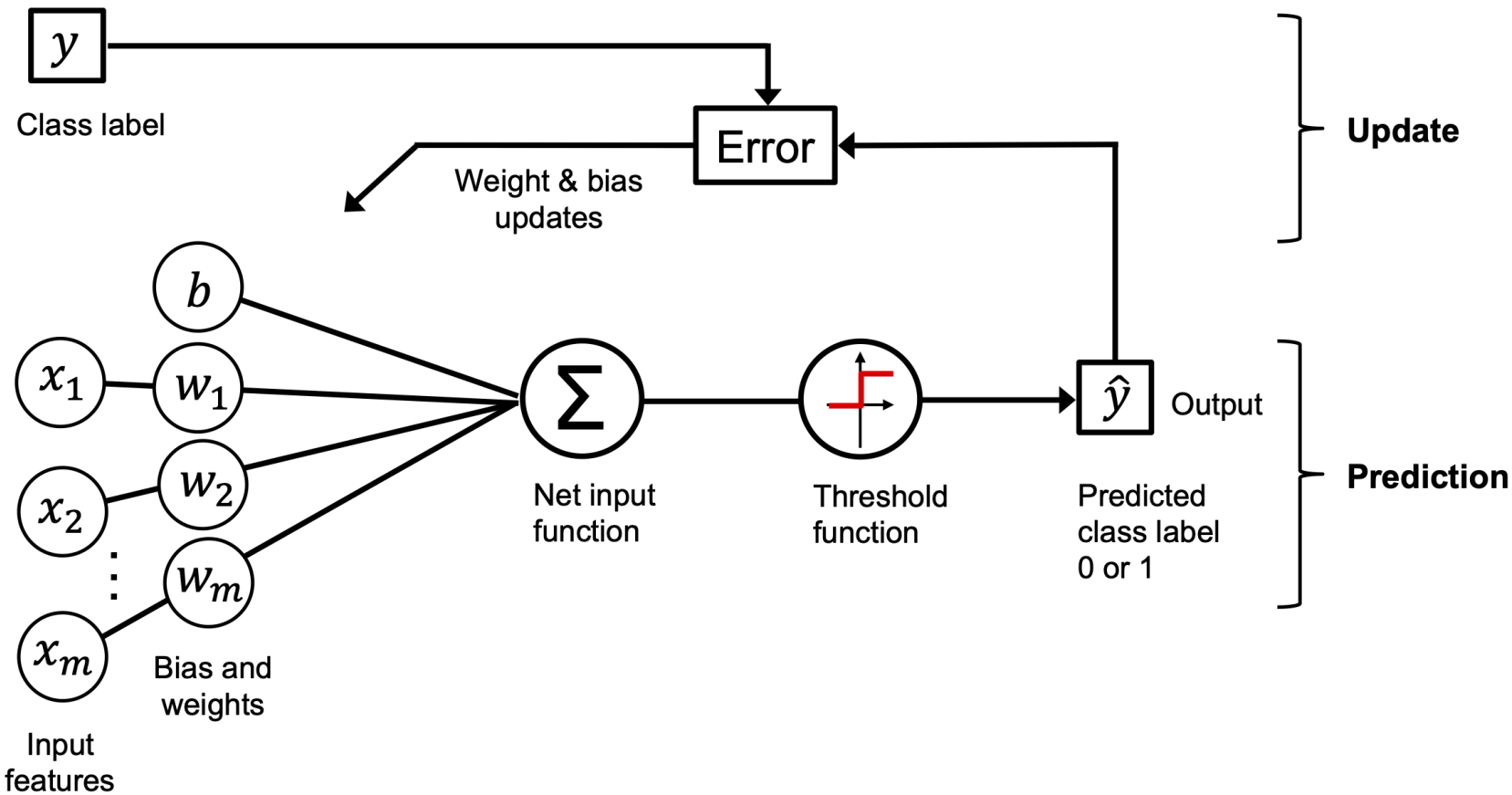
$$\Delta w_2 = 1 \cdot 3 = 3$$

$$\Delta b = 1$$

2.1.2 퍼셉트론 학습 규칙

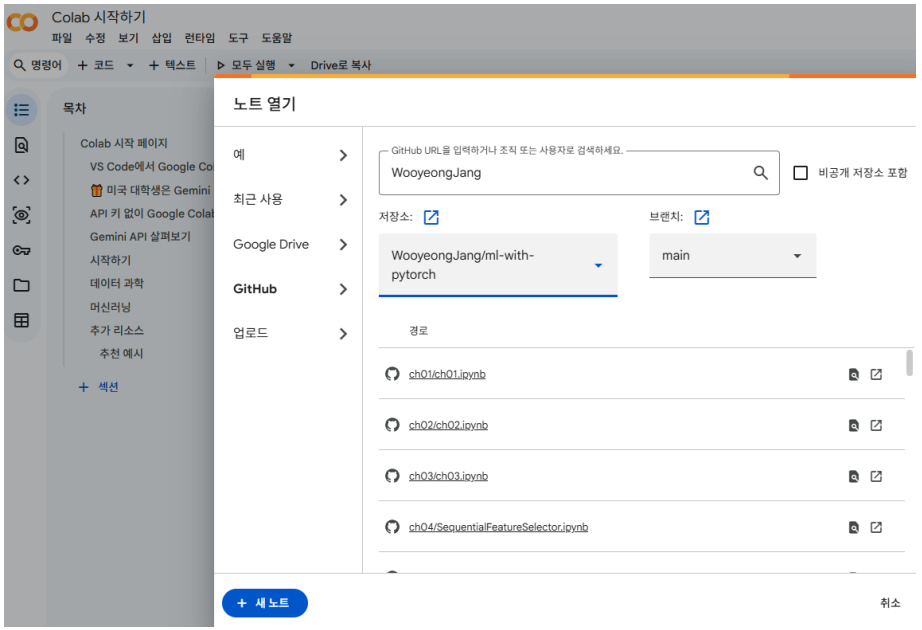


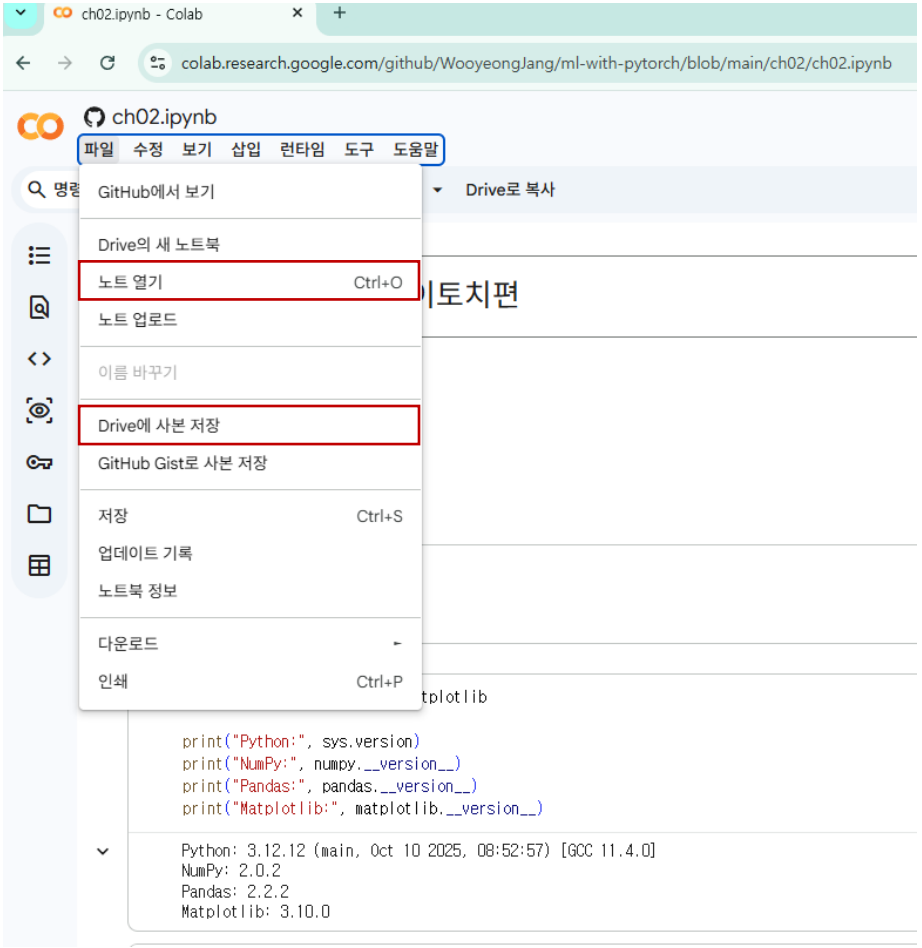
2.1.2 퍼셉트론 학습 규칙



2.2 파이썬으로 퍼셉트론 학습 알고리즘 구현

https://colab.research.google.com





2.2 파이썬으로 퍼셉트론 학습 알고리즘 구현

수업 환경 확인

- 권장 패키지 버전

```
'numpy': '1.21.2'
'pandas': '1.3.2'
'matplotlib': '3.4.3'
```

[1] 0초

```python
import sys, numpy, pandas, matplotlib

print("Python:", sys.version)
print("NumPy:", numpy.__version__)
print("Pandas:", pandas.__version__)
print("Matplotlib:", matplotlib.__version__)
```

```
Python: 3.12.12 (main, Oct 10 2025, 08:52:57) [GCC 11.4.0]
NumPy: 2.0.2
Pandas: 2.2.2
Matplotlib: 3.10.0
```

[2] 0초

```python
# sep 기본 옵션 ' ' 공백 한칸
print("A","B")
print("A","B", sep='|')
print("A","B", sep='')
# end 기본 옵션 ₩n  줄바꿈
print("A","B", end ='')
print("A","B", sep='|', end='')
print("A","B", sep='')
```

```
A B
A|B
AB
A BA|BAB
```

[8] 2초

```python
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
!apt-get install -y fonts-nanum

plt.rcParams['font.family'] = 'NanumGothic'
plt.rcParams['axes.unicode_minus'] = False
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
fonts-nanum is already the newest version (20200506-1).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
```

2.2 파이썬으로 퍼셉트론 학습 알고리즘 구현

객체를 만들기 위한 설계도

- 퍼셉트론 기계 설계
  1. 처음에 설정이 필요하다
  2. 데이터를 보고 학습해야 한다
  3. 새로운 데이터를 예측해야 한다
  4. 내부 점수를 계산해야 한다
- Class

```
Perceptron ()
  ├── __init__() → 처음 설정하는 기능
  ├── fit()      → 학습하는 기능
  ├── predict()  → 예측하는 기능
  └── net_input()→ 점수 계산 기능
```

형태	예시	의미	강제성	사용 목적
`_`	`_`	임시 변수	없음	의미 없는 값 저장
`_var`	`_weight`	내부용 변수(관례상 private)	약함	외부에서 쓰지 말라는 신호
`var_`	`w_`, `coef_`	학습 후 생성된 속성 (sklearn 스타일)	관례	매개변수와 구분
`__var`	`__weight`	이름 맹글링 (name mangling)	강함	클래스 내부 보호
`__x__`	`__init__`, `__str__`	파이썬 기본(매직) 메서드	매우 강함	파이썬이 자동 호출

2.2.1 객체 지향 퍼셉트론 API

```python
import numpy as np


class Perceptron:
    """퍼셉트론 분류기

    매개변수
    ------------
    eta : float
      학습률 (0.0과 1.0 사이)
    n_iter : int
      훈련 데이터셋 반복 횟수
    random_state : int
      가중치 무작위 초기화를 위한 난수 생성기 시드

    속성
    -----------
    w_ : 1d-array
      학습된 가중치
    b_ : 스칼라
      학습된 절편 유닛
    errors_ : list
      에포크마다 누적된 분류 오류

    """
```

```python
print(Perceptron.__doc__)
```

```
퍼셉트론 분류기

    매개변수
    ------------
    eta : float
      학습률 (0.0과 1.0 사이)
    n_iter : int
      훈련 데이터셋 반복 횟수
    random_state : int
      가중치 무작위 초기화를 위한 난수 생성기 시드

    속성
    -----------
    w_ : 1d-array
      학습된 가중치
    b_ : 스칼라
      학습된 절편 유닛
    errors_ : list
      에포크마다 누적된 분류 오류
```

2.2.1 객체 지향 퍼셉트론 API

```
def __init__(self, eta=0.01, n_iter=50, random_state=1):
    self.eta = eta
    self.n_iter = n_iter
    self.random_state = random_state
```

2.2.1 객체 지향 퍼셉트론 API

```
def fit(self, X, y):
    """훈련 데이터 학습

    매개변수
    ----------
    X : array-like, shape = [n_samples, n_features]
      n_samples개의 샘플과 n_features개의 특성으로 이루어진 훈련 데이터
    y : array-like, shape = [n_samples]
      타깃값

    반환값
    -------
    self : object
    """
    rgen = np.random.RandomState(self.random_state)
    self.w_ = rgen.normal(loc=0.0, scale=0.01, size=X.shape[1])
    self.b_ = np.float64(0.) # np.float_ is deprecated in NumPy 2.0

    self.errors_ = []

    for _ in range(self.n_iter):
        errors = 0
        for xi, target in zip(X, y):
            update = self.eta * (target - self.predict(xi))
            self.w_ += update * xi
            self.b_ += update
            errors += int(update != 0.0)
        self.errors_.append(errors)
    return self
```

2.2.1 객체 지향 퍼셉트론 API

```
def net_input(self, X):
    """입력 계산"""
    return np.dot(X, self.w_) + self.b_

def predict(self, X):
    """단위 계단 함수를 사용하여 클래스 레이블을 반환합니다"""
    return np.where(self.net_input(X) >= 0.0, 1, 0)
```

2.2.1 객체 지향 퍼셉트론 API

```python
class Perceptron:
    """간단한 퍼셉트론 분류기"""

    def __init__(self, eta=0.1, n_iter=50):
        """학습률과 반복 횟수 설정"""
        self.eta = eta
        self.n_iter = n_iter

    def fit(self, X, y):
        """데이터로 가중치 학습"""
        self.w_ = np.zeros(X.shape[1])
        self.b_ = 0.0
        self.errors_ = []

        for _ in range(self.n_iter):
            errors = 0
            for xi, target in zip(X, y):
                update = self.eta * (target - self.predict(xi))
                self.w_ += update * xi
                self.b_ += update
                if update != 0.0:
                    errors += 1
            self.errors_.append(errors)

        return self
```

```python
    def net_input(self, X):
        """가중합 계산"""
        return np.dot(X, self.w_) + self.b_

    def predict(self, X):
        """클래스 예측"""
        return np.where(self.net_input(X) >= 0.0, 1, -1)
```

2.2.2 붓꽃 데이터셋에서 퍼셉트론 훈련

붓꽃 데이터셋 읽기

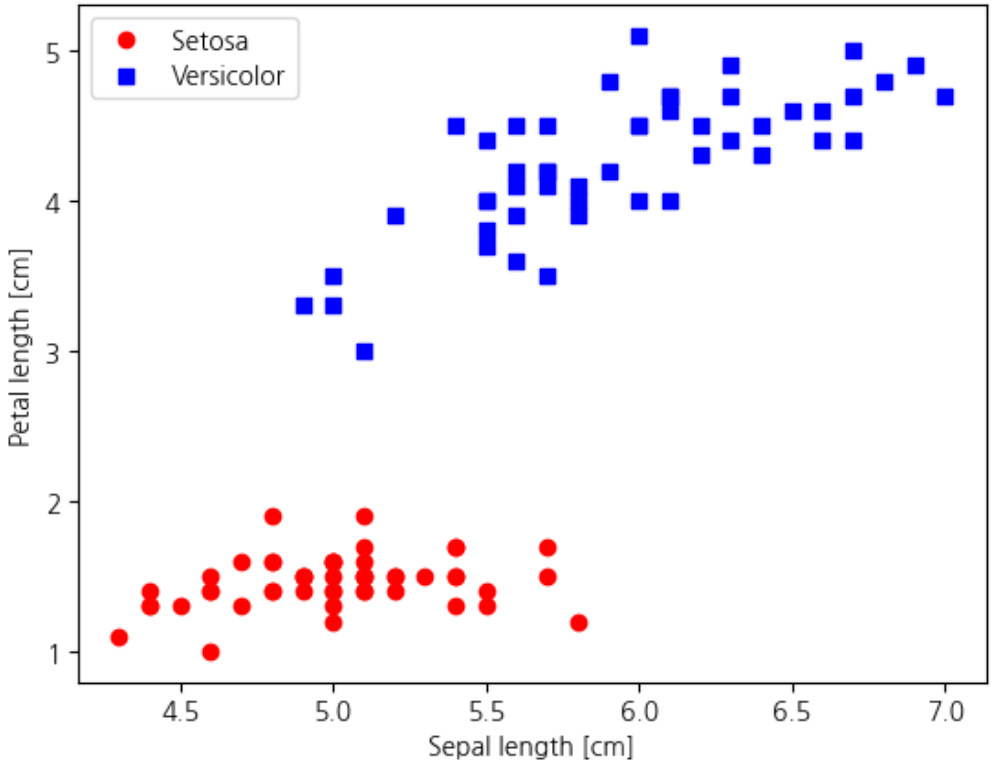
```
import os
import pandas as pd

try:
    s = 'https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data'
    print('From URL:', s)
    df = pd.read_csv(s,
                     header=None,
                     encoding='utf-8')

except HTTPError:
    s = 'iris.data'
    print('From local Iris path:', s)
    df = pd.read_csv(s,
                     header=None,
                     encoding='utf-8')

df.tail()
```

From URL: https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data

	0	1	2	3	4
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

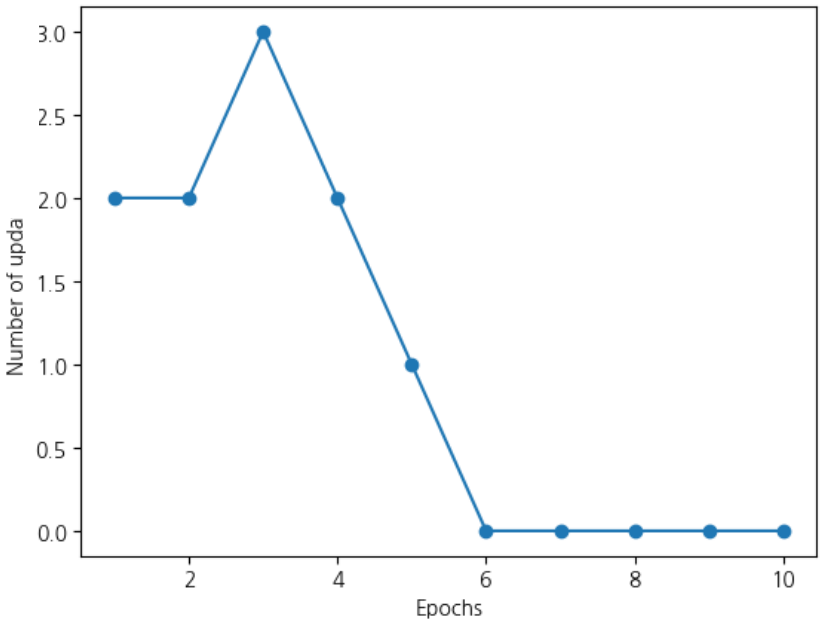
2.2.2 붓꽃 데이터셋에서 퍼셉트론 훈련

```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np

# setosa와 versicolor를 선택합니다
y = df.iloc[0:100, 4].values
y = np.where(y == 'Iris-setosa', 0, 1)

# 꽃받침 길이와 꽃잎 길이를 추출합니다
X = df.iloc[0:100, [0, 2]].values

# 산점도를 그립니다
plt.scatter(X[:50, 0], X[:50, 1],
            color='red', marker='o', label='Setosa')
plt.scatter(X[50:100, 0], X[50:100, 1],
            color='blue', marker='s', label='Versicolor')

plt.xlabel('Sepal length [cm]')
plt.ylabel('Petal length [cm]')
plt.legend(loc='upper left')

# plt.savefig('images/02_06.png', dpi=300)
plt.show()
```



2.2.2 붓꽃 데이터셋에서 퍼셉트론 훈련

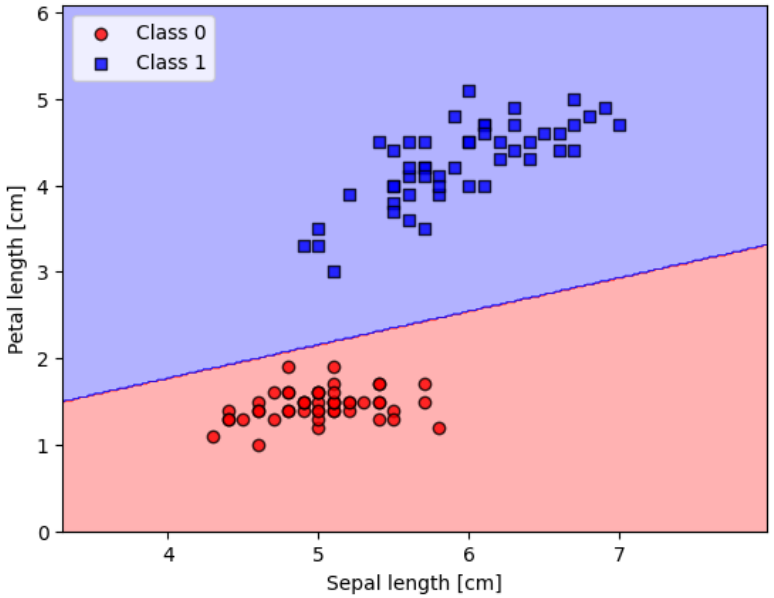
```
ppn = Perceptron(eta=0.1, n_iter=10)

ppn.fit(X, y)

plt.plot(range(1, len(ppn.errors_) + 1), ppn.errors_, marker='o')
plt.xlabel('Epochs')
plt.ylabel('Number of updates')

# plt.savefig('images/02_07.png', dpi=300)
plt.show()
```



2.2.2 붓꽃 데이터셋에서 퍼셉트론 훈련

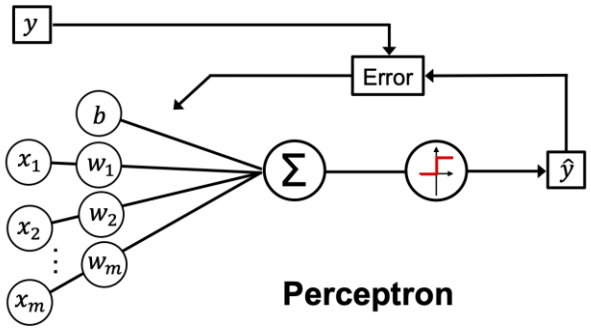
```python
from matplotlib.colors import ListedColormap


def plot_decision_regions(X, y, classifier, resolution=0.02):

    # 마커와 컬러맵을 설정합니다
    markers = ('o', 's', '^', 'v', '<')
    colors = ('red', 'blue', 'lightgreen', 'gray', 'cyan')
    cmap = ListedColormap(colors[:len(np.unique(y))])

    # plot the decision surface
    x1_min, x1_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    x2_min, x2_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, resolution),
                           np.arange(x2_min, x2_max, resolution))
    lab = classifier.predict(np.array([xx1.ravel(), xx2.ravel()]).T)
    lab = lab.reshape(xx1.shape)
    plt.contourf(xx1, xx2, lab, alpha=0.3, cmap=cmap)
    plt.xlim(xx1.min(), xx1.max())
    plt.ylim(xx2.min(), xx2.max())

    # 샘플의 산점도를 그립니다
    for idx, cl in enumerate(np.unique(y)):
        plt.scatter(x=X[y == cl, 0],
                    y=X[y == cl, 1],
                    alpha=0.8,
                    c=colors[idx],
                    marker=markers[idx],
                    label=f'Class {cl}',
                    edgecolor='black')
```

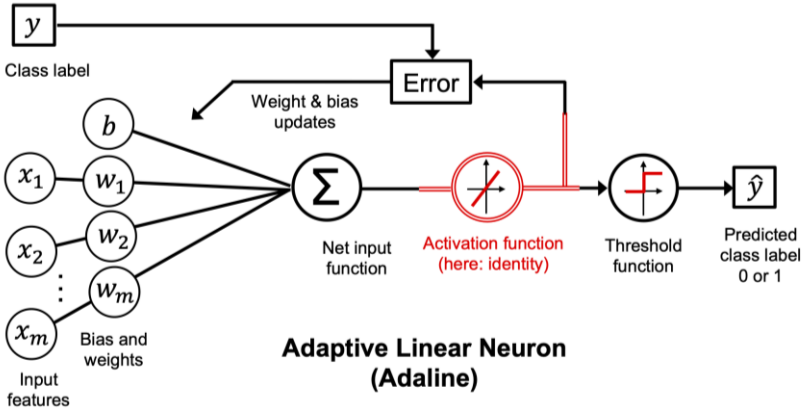
```python
plot_decision_regions(X, y, classifier=ppn)
plt.xlabel('Sepal length [cm]')
plt.ylabel('Petal length [cm]')
plt.legend(loc='upper left')


#plt.savefig('images/02_08.png', dpi=300)
plt.show()
```



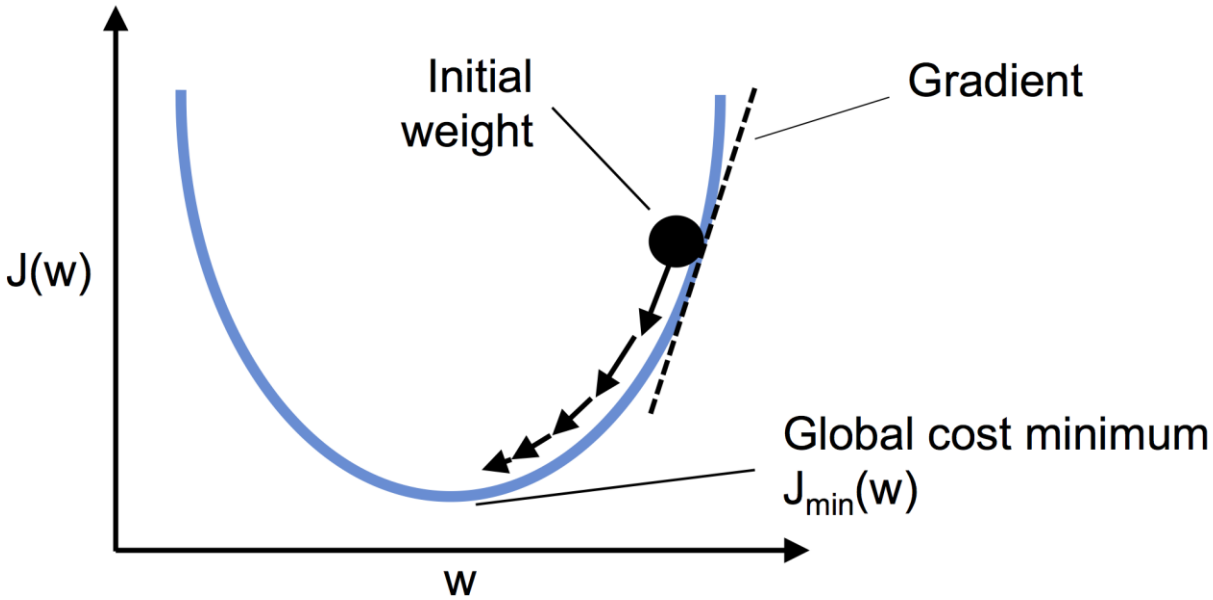
2.3 적응형 선형 뉴런과 학습의 수렴

- 적응형 선형 뉴런(ADAptive Linear Neuron, ADALINE)
- 퍼셉트론 알고리즘 이후 아달린 알고리즘 발표
- 아달린은 퍼셉트론의 향상된 버전
- 퍼셉트론은 맞았냐 틀렸냐만 보지만, 아달린은 얼마나 틀렸는지를 계산하고, 그 오차를 경사하강법으로 최소화





2.3.1 경사 하강법으로 손실 함수 최소화

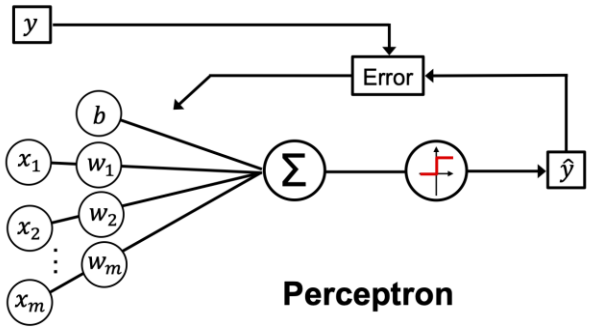


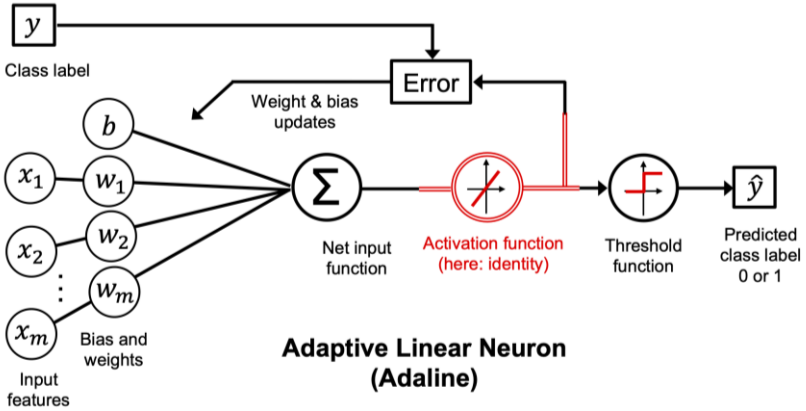
	퍼셉트론	아달린
출력	계단 함수	선형 출력
학습 기준	분류 결과	오차
손실 함수	없음	MSE
최적화	단순 업데이트	경사하강법

2.3.2 파이썬으로 아달린 구현

Perceptron()
- ├── __init__()
- ├── fit()
- ├── net_input()
- └── predict()

Adaline()
- ├── __init__()
- ├── fit()
- ├── net_input()
- ├── activation()
- └── predict()





2.3.2 파이썬으로 아달린 구현

```python
import numpy as np


class Adaline:

    def __init__(self, eta=0.01, n_iter=50, random_seed=1):
        """초기 설정"""

        self.eta = eta                  # 학습률 (learning rate)
        self.n_iter = n_iter            # 학습 반복 횟수 (epoch)
        self.random_seed = random_seed # 난수 고정 (결과 재현 가능)

    def fit(self, X, y): ···

    def net_input(self, X):
        """선형 결합 계산 z = w^T x + b"""

        return np.dot(X, self.w_) + self.b_

    def activation(self, z):
        """Adaline의 활성화 함수 (선형 함수)"""

        return z

    def predict(self, X):
        """최종 예측 (0 또는 1로 분류)"""

        return np.where(self.activation(self.net_input(X)) >= 0.0, 1, 0)
```

```python
import numpy as np


class Adaline:

    def __init__(self, eta=0.01, n_iter=50, random_seed=1): ···

    def fit(self, X, y):
        """데이터를 이용해 가중치 학습"""
        rgen = np.random.RandomState(self.random_seed)  # 난수 생성기 생성
        # 가중치를 0에 가까운 작은 난수로 초기화
        self.w_ = rgen.normal(loc=0.0, scale=0.01, size=X.shape[1])
        self.b_ = 0.0                   # 절편(bias) 초기화
        self.losses_ = []               # epoch마다 손실 저장
        for _ in range(self.n_iter):    # epoch 반복
            net_input = self.net_input(X)      # 선형 결합 계산
            output = self.activation(net_input) # 활성화 함수 적용
            errors = y - output         # 실제값 - 예측값 (오차 계산)
            # 경사하강법을 이용한 가중치 업데이트
            self.w_ += self.eta * X.T.dot(errors)
            # bias 업데이트
            self.b_ += self.eta * errors.sum()
            # MSE 손실 계산
            loss = (errors ** 2).mean() / 2.0
            # 손실 기록
            self.losses_.append(loss)
        return self

    def net_input(self, X): ···

    def activation(self, z): ···

    def predict(self, X): ···
```

2.4 요약

- 지도 학습의 기초적인 선형 분류기 개념
- 퍼셉트론과 아달린 알고리즘 구현